

System Overview

The Consultancy Appointment and Management System (CAMS) is a web-based database application developed for CS306 Project Phase 4. The system manages clients, professionals, services, locations, appointments, payments, and reviews. MySQL is used for the relational database operations, triggers, and stored procedures. MongoDB is used for the support ticket system.

The application has two main interfaces:

- User interface: Used for testing triggers, calling stored procedures, and creating support tickets.
- Admin interface: Used for managing MongoDB support tickets, adding admin comments, and marking tickets as resolved.

The system is implemented using PHP, MySQL, MongoDB, and XAMPP.

Trigger 1

This trigger prevents inserting a review with an invalid rating. Rating must be between 1 and 5.

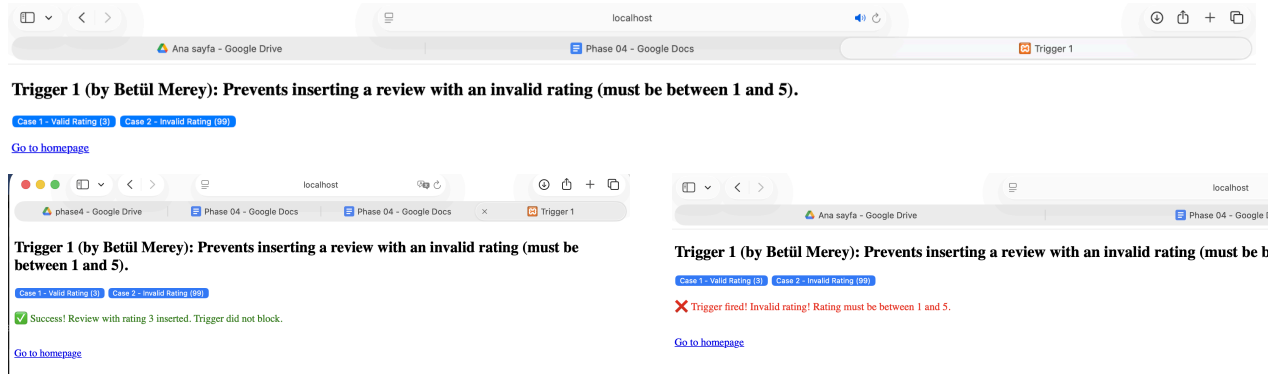
```
DELIMITER $$
```

```
CREATE TRIGGER before_review_insert
BEFORE INSERT ON REVIEW
FOR EACH ROW
BEGIN
    IF NEW.rating < 1 OR NEW.rating > 5 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Invalid rating! Rating must be between 1 and 5.';
    END IF;
END$$
```

```
DELIMITER ;
```

Cases:

- **Case 1 - Valid Rating (3):** Inserts a review with rating 3. Trigger does not block the operation. Expected output: Success message.
- **Case 2 - Invalid Rating (99):** Attempts to insert a review with rating 99. Trigger fires and blocks the operation. Expected output: Error message.



Trigger 2

This trigger prevents deleting a client who has existing appointments in the system

DELIMITER \$\$

```
CREATE TRIGGER before_client_delete
BEFORE DELETE ON CLIENT
FOR EACH ROW
BEGIN
    DECLARE appointment_count INT;

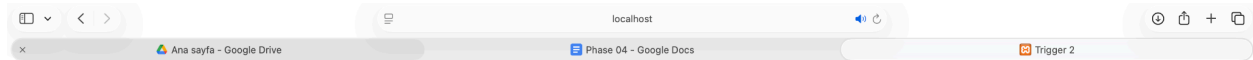
    SELECT COUNT(*) INTO appointment_count
    FROM APPOINTMENT
    WHERE client_id = OLD.client_id;

    IF appointment_count > 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Cannot delete client! Client has existing
appointments.';
    END IF;
END$$
```

DELIMITER ;

Cases:

- **Case 1 - Delete Client Without Appointments (ID:6):** A temporary client with no appointments is inserted and then deleted. Trigger does not block. Expected output: Success message.
- **Case 2 - Delete Client With Appointments (ID:1):** Attempts to delete Alice Johnson who has existing appointments. Trigger fires and blocks the deletion. Expected output: Error message.



Trigger 2 (by Nehir Ceylan): Prevents deleting a client who has existing appointments.

[Case 1 - Delete Client Without Appointments \(ID:6\)](#) [Case 2 - Delete Client With Appointments \(ID:1\)](#)

[Go to homepage](#)



Trigger 2 (by Nehir Ceylan): Prevents deleting a client who has existing appointments.

[Case 1 - Delete Client Without Appointments \(ID:6\)](#) [Case 2 - Delete Client With Appointments \(ID:1\)](#)

✓ Success! Client with no appointments deleted. Trigger did not block.

[Go to homepage](#)



Trigger 2 (by Nehir Ceylan): Prevents deleting a client who has existing appointments.

[Case 1 - Delete Client Without Appointments \(ID:6\)](#) [Case 2 - Delete Client With Appointments \(ID:1\)](#)

✗ Trigger fired! Cannot delete client! Client has existing appointments.

[Go to homepage](#)

Stored Procedure 1

Returns all appointments and related details for a given client

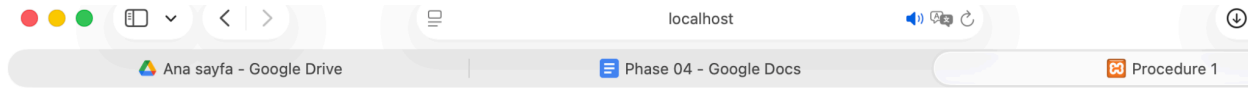
DELIMITER \$\$

```
CREATE PROCEDURE GetClientAppointments(IN p_client_id INT)
BEGIN
    SELECT
        A.appointment_id,
        A.date,
        P.name AS professional_name,
        S.service_name,
        L.address,
        L.location_type
    FROM APPOINTMENT A
    JOIN PROFESSIONAL P ON A.professional_id = P.professional_id
    JOIN SERVICE S ON A.service_id = S.service_id
    JOIN LOCATION L ON A.location_id = L.location_id
    WHERE A.client_id = p_client_id;
END$$
```

DELIMITER ;

Input Parameters:

- **client_id:** The ID of the client



Stored Procedure 1 (by Betül Meray): Lists all appointments and details for a given client.

Enter Client ID (1-5)

Call Procedure

Appointment ID	Date	Professional	Service	Address	Location Type
1	2024-01-10	Dr. Sarah White	Career Counseling	123 Main St, New York	Physical
6	2024-03-10	Mr. James Brown	Financial Planning	Online - Zoom	Online

[Go to homepage](#)

- Trial Input id is 1

Stored Procedure 2

Calculates the average rating of a given professional based on reviews.

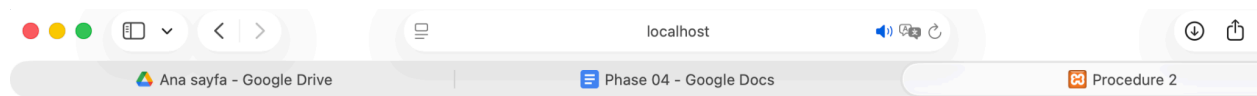
DELIMITER \$\$

```
CREATE PROCEDURE GetProfessionalAvgRating(IN p_professional_id INT)
BEGIN
    SELECT
        P.name AS professional_name,
        P.specialization,
        AVG(R.rating) AS average_rating,
        COUNT(R.review_id) AS total_reviews
    FROM PROFESSIONAL P
    JOIN APPOINTMENT A ON P.professional_id = A.professional_id
    JOIN REVIEW R ON A.appointment_id = R.appointment_id
    WHERE P.professional_id = p_professional_id
    GROUP BY P.name, P.specialization;
END$$
```

DELIMITER ;

Input Parameters:

- **professional_id:** The ID of the professional



Stored Procedure 2 (by Nehir Ceylan): Calculates the average rating of a given professional.

Enter Professional ID (1-5) [Call Procedure](#)

Professional Name	Specialization	Average Rating	Total Reviews
Dr. Sarah White	Career Counseling	4.0000	3

[Go to homepage](#)

- Trial Input id is 1

Stored Procedure 3

List all appointments that fall within a given date range

```
DELIMITER $$
```

```
CREATE PROCEDURE GetAppointmentsByDateRange(IN p_start_date DATE, IN  
p_end_date DATE)
```

```
BEGIN
```

```
    SELECT
```

```
        A.appointment_id,
```

```
        A.date,
```

```
        C.name AS client_name,
```

```
        P.name AS professional_name,
```

```
        S.service_name
```

```
    FROM APPOINTMENT A
```

```
    JOIN CLIENT C ON A.client_id = C.client_id
```

```
    JOIN PROFESSIONAL P ON A.professional_id = P.professional_id
```

```
    JOIN SERVICE S ON A.service_id = S.service_id
```

```
    WHERE A.date BETWEEN p_start_date AND p_end_date
```

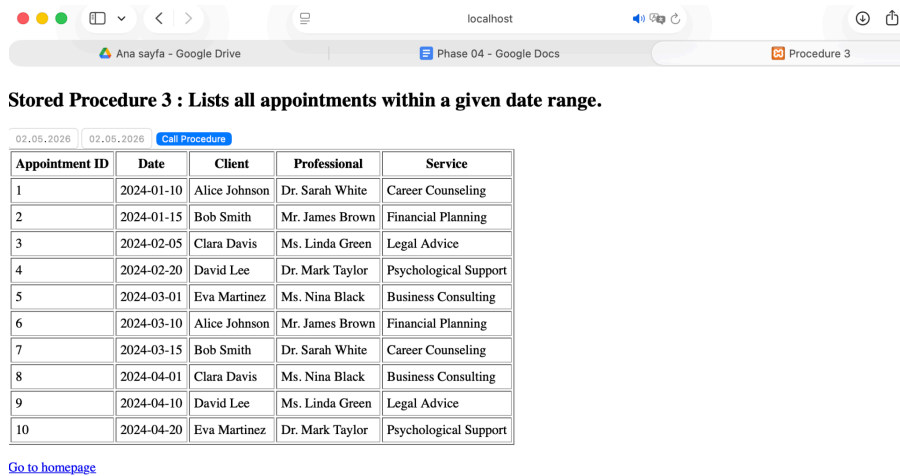
```
    ORDER BY A.date ASC;
```

```
END$$
```

```
DELIMITER ;
```

Input Parameters:

- **start_date:** Start date of the range (date format: YYYY-MM-DD)
- **end_date:** End date of the range (date format: YYYY-MM-DD)



The screenshot shows a web browser window with a localhost address bar. Below the address bar, there are tabs for 'Ana sayfa - Google Drive', 'Phase 04 - Google Docs', and 'Procedure 3'. The main content area displays a date range selector with '02.05.2026' selected and a 'Call Procedure' button. Below this is a table with 5 columns: Appointment ID, Date, Client, Professional, and Service. The table contains 10 rows of appointment data.

Appointment ID	Date	Client	Professional	Service
1	2024-01-10	Alice Johnson	Dr. Sarah White	Career Counseling
2	2024-01-15	Bob Smith	Mr. James Brown	Financial Planning
3	2024-02-05	Clara Davis	Ms. Linda Green	Legal Advice
4	2024-02-20	David Lee	Dr. Mark Taylor	Psychological Support
5	2024-03-01	Eva Martinez	Ms. Nina Black	Business Consulting
6	2024-03-10	Alice Johnson	Mr. James Brown	Financial Planning
7	2024-03-15	Bob Smith	Dr. Sarah White	Career Counseling
8	2024-04-01	Clara Davis	Ms. Nina Black	Business Consulting
9	2024-04-10	David Lee	Ms. Linda Green	Legal Advice
10	2024-04-20	Eva Martinez	Dr. Mark Taylor	Psychological Support

[Go to homepage](#)

- Trial inputs: Start date: 01-01-2024 End Date: 12-31-2024 -

MongoDB Support Ticket System

Workflow:

The support ticket system is built using MongoDB to store and manage ticket data. The system works as follows:

1. **Ticket Creation (User Side):** A user enters their username and a message body, then submits the form. The ticket is stored in MongoDB with status set to true (active), an empty comments array, and the creation timestamp.
2. **Ticket Listing (User Side):** Users select their username from a dropdown menu. Only users with at least one active ticket appear in the dropdown. After selecting, their active tickets are displayed with a "View Details" link.
3. **Ticket Detail (User Side):** Users can view ticket details including status, body, creation date, and all comments. Users can also add new comments to the ticket.
4. **Admin Ticket Management:** Admins can view all active tickets from all users. Admins can add comments (automatically assigned as "admin") and mark tickets as resolved by clicking "Deactivate Ticket", which sets the status to false. Deactivated tickets no longer appear in either the user or admin views

Implementation:

In our project, MongoDB is used to implement the support ticket system. Each ticket is stored in MongoDB with the following fields: username, body, created_at, status, and comments array.

1. MongoDB Connection

The system connects to MongoDB using the following PHP code:

```
$manager = new MongoDB\Driver\Manager("mongodb://localhost:27017");  
  
$namespace = "cams.tickets";
```

2. Create Ticket (User Side)

A new ticket is created when a user submits the form.
The ticket is inserted into MongoDB with default values.

```
$bulk = new MongoDB\Driver\BulkWrite();  
  
$bulk->insert([  
    'username' => $username,  
    'body'      => $body,  
    'created_at' => date('Y-m-d H:i:s'),  
    'status'    => true,  
    'comments'  => []  
]);  
  
$manager->executeBulkWrite($namespace, $bulk);
```

3. List Active Tickets

This query retrieves all active tickets from MongoDB.

```
$filter = ['status' => true];  
  
$query = new MongoDB\Driver\Query($filter);  
  
$cursor = $manager->executeQuery($namespace, $query);
```

4. Filter Tickets by Username

Users can view only their own active tickets.

```
$filter = [  
  
    'username' => $selected_user,  
  
    'status' => true  
  
];  
  
$query = new MongoDB\Driver\Query($filter);  
  
$cursor = $manager->executeQuery($namespace, $query);
```

5. View Ticket Details

A specific ticket is retrieved using its unique MongoDB ObjectId.

```
$filter = ['_id' => new MongoDB\BSON\ObjectId($ticket_id)];  
  
$query = new MongoDB\Driver\Query($filter);  
  
$cursor = $manager->executeQuery($namespace, $query);
```


6. Add Comment to Ticket

Users and admins can add comments to a ticket.

Comments are stored in an array using the **\$push** operator.

```
$newComment = [  
    'username' => $username,  
    'comment' => $comment,  
    'created_at' => date('Y-m-d H:i:s')  
];  
  
$bulk = new MongoDB\Driver\BulkWrite();  
  
$bulk->update(  
    ['_id' => new MongoDB\BSON\ObjectId($ticket_id)],  
    ['$push' => ['comments' => $newComment]]  
);  
  
$manager->executeBulkWrite($namespace, $bulk);
```

7. Admin: List All Active Tickets

The admin panel displays all active tickets.

```
$filter = ['status' => true];  
  
$query = new MongoDB\Driver\Query($filter);  
  
$cursor = $manager->executeQuery($namespace, $query);  
  
$tickets = $cursor->toArray();
```

8. Admin: Deactivate Ticket

Admins can mark a ticket as resolved by setting its status to false.

```
$bulk = new MongoDB\Driver\BulkWrite();

$bulk->update(

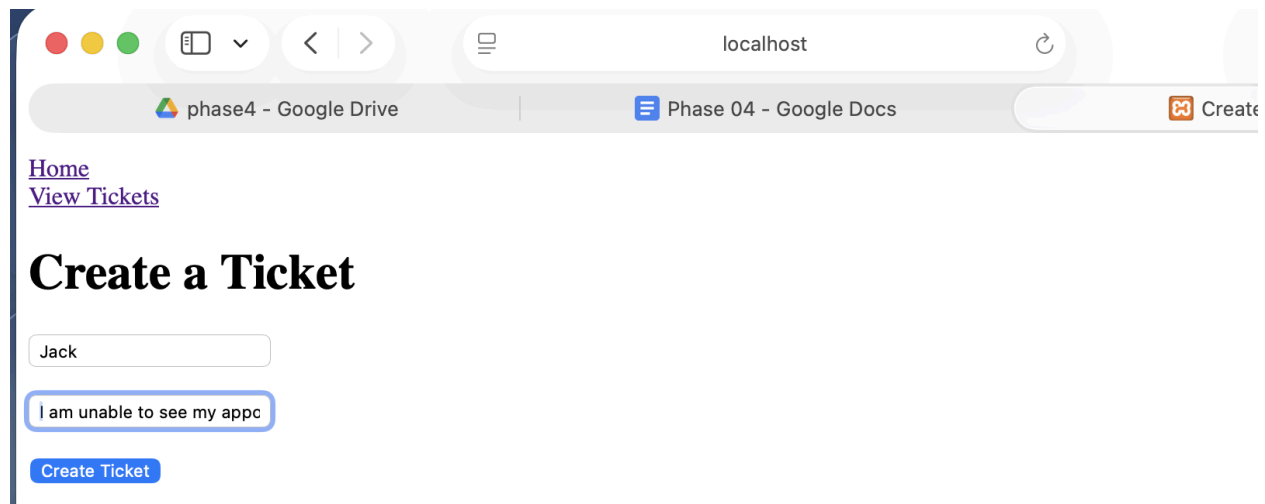
['_id' => new MongoDB\BSON\ObjectId($ticket_id)],

['$set' => ['status' => false]]

);

$manager->executeBulkWrite($namespace, $bulk);
```

Ticket Creation Page



phase4 - Google Drive | Phase 04 - Google Docs | Create

[Home](#)
[View Tickets](#)

Create a Ticket

Jack

I am unable to see my appc

Create Ticket

Ticket List Page

📄

🔍

🏠

localhost

🔄

phase4 - Google Drive

Phase 04 - Google Docs

Support

[Home](#)

Jack [Select](#)

[Create a Ticket](#)

Results:

Status: Active

Body: I am unable to see my appointments

Created At: 2026-05-09 20:55:34

Username: Jack

[View Details](#)

Ticket Detail Page

Ticket Details

Username: Jack

Body: I am unable to see my appointments

Status: Active

Created At: 2026-05-09 20:55:34

Comments:

[Add Comment](#)

[Back to Tickets](#)

Admin Panel

localhost

phase4 - Google Drive | Phase 04 - Google Docs | Admin

Admin Panel - Active Tickets

Results:

Status: Active
Body: I cannot see my appointment details
Created At: 2026-05-06 13:45:05
Username: betul
[View Details](#)

Status: Active
Body: I have a payment issue
Created At: 2026-05-06 13:45:16
Username: nehir
[View Details](#)

Status: Active
Body: Please reschedule my appointment
Created At: 2026-05-06 13:45:26
Username: alice
[View Details](#)

Status: Active
Body: I am unable to see my appointments
Created At: 2026-05-09 20:55:34
Username: Jack
[View Details](#)

localhost

phase4 - Google Drive | Phase 04 - Google Docs | Admin Panel | Admin - Ticket Details

Deactivate Ticket

Ticket Details

Username: Jack
Body: I am unable to see my appointments
Status: Active
Created At: 2026-05-09 20:55:34

Comments:

Add a comment

Add Comment

[Back to Tickets](#)

Admin Ticket Details

The screenshot displays the MongoDB Compass interface. On the left, the 'Compass' sidebar shows a tree view of the database structure under 'localhost:27017', including collections like 'admin', 'cams', 'tickets', 'config', and 'local'. The 'tickets' collection is selected. The main panel shows the 'tickets' collection with 5 documents. The first document is expanded, showing the following fields:

```
{
  "_id": ObjectId('69f643a65309defe810f29e1'),
  "username": "alice",
  "body": "need help with appointment",
  "created_at": "2026-05-02 20:34:14",
  "status": false,
  "comments": Array (1)
}
```

Challenges and Design Decisions

MongoDB Installation on macOS: The main challenge of this project was installing the MongoDB PHP driver on macOS with XAMPP. Since XAMPP uses its own PHP version, the standard PECL installation failed due to missing autoconf and OpenSSL libraries. The issue was resolved by installing autoconf and OpenSSL via Homebrew before running the PECL install command. Additionally, Composer had to be run using XAMPP's own PHP executable instead of the system PHP to ensure version compatibility.

Database Design Decision: MongoDB was chosen for the support ticket system because its flexible document structure is well-suited for storing tickets with a variable number of comments. Unlike MySQL, MongoDB allows comments to be stored directly inside the ticket document as an array, which simplifies the data model.

Trigger Design: For Trigger 1, the review_id was made random using PHP's rand() function to avoid duplicate primary key errors during repeated testing. This ensures the trigger can be tested multiple times without manually cleaning the database.

Assumption: It was assumed that the CAMS database is already populated with the mock data from Phase 3. The triggers and stored procedures were tested using this existing data.

Conclusion

In Phase 4, the Consultancy Appointment and Management System (CAMS) was extended into a fully functional web application. The project integrates MySQL triggers, MySQL stored procedures, and MongoDB support ticket operations into a PHP-based user and admin interface.

The application demonstrates relational database consistency through triggers and stored procedures, while also using MongoDB for flexible ticket management. The trigger implementations protect data integrity by preventing invalid rating insertions and blocking deletion of clients with existing appointments. The stored procedures simplify complex database operations by centralizing appointment retrieval and rating calculations.

The final system provides a complete interface for testing database functionality and managing support tickets through both user and admin panels.

AI Usage Statement

In this project, Claude (Anthropic) was used as an AI assistance tool. AI support was used during the preparation of the web interface and the project document. Specifically, Claude helped with writing and organizing parts of the HTML/PHP interface structure, resolving MongoDB installation issues on macOS, and preparing some explanatory text for the documentation.

The database topic, system idea, ER design, relational schema, SQL implementation, triggers, stored procedures, and core project logic were designed and developed by the group members. AI was not used to create the database design or the main project idea. All AI-assisted content was reviewed, edited, tested, and finalized by the group before submission.